Solution: Sqrt

Problem Statement

Given a non-negative integer `x`, return the square root of `x` rounded down to the nearest integer. The returned integer should be non-negative as well.

You **must not use** any built-in exponent function or operator. For example, do not use `pow(x, 0.5)` in C++ or `x ** 0.5` in Python.

Example 1:

```
Input: x = 8
Output: 2
Explanation: The square root of 8 is 2.8284, and since we need to return
the floor of the square root (integer), hence we returned 2.
```

Example 2:

```
Input: x = 4
Output: 2
Explanation: The square root of 4 is 2.
```

Example 3:

```
Input: x = 2
Output: 1
Explanation: The square root of 2 is 1.414, and since we need to return
the floor of the square root (integer), hence we returned 1.
```

Solution

We can follow the Binary Search approach to calculate the square root of an integer `x` without using any in-built `sqrt` function.

Since the square root of a number `x` lies between `0` and `x/2` for all `x >= 0`, we can use binary search within this range (`0 to x/2`) to find the square root. The integer part (i.e., the floor) of the square root will be the final result.

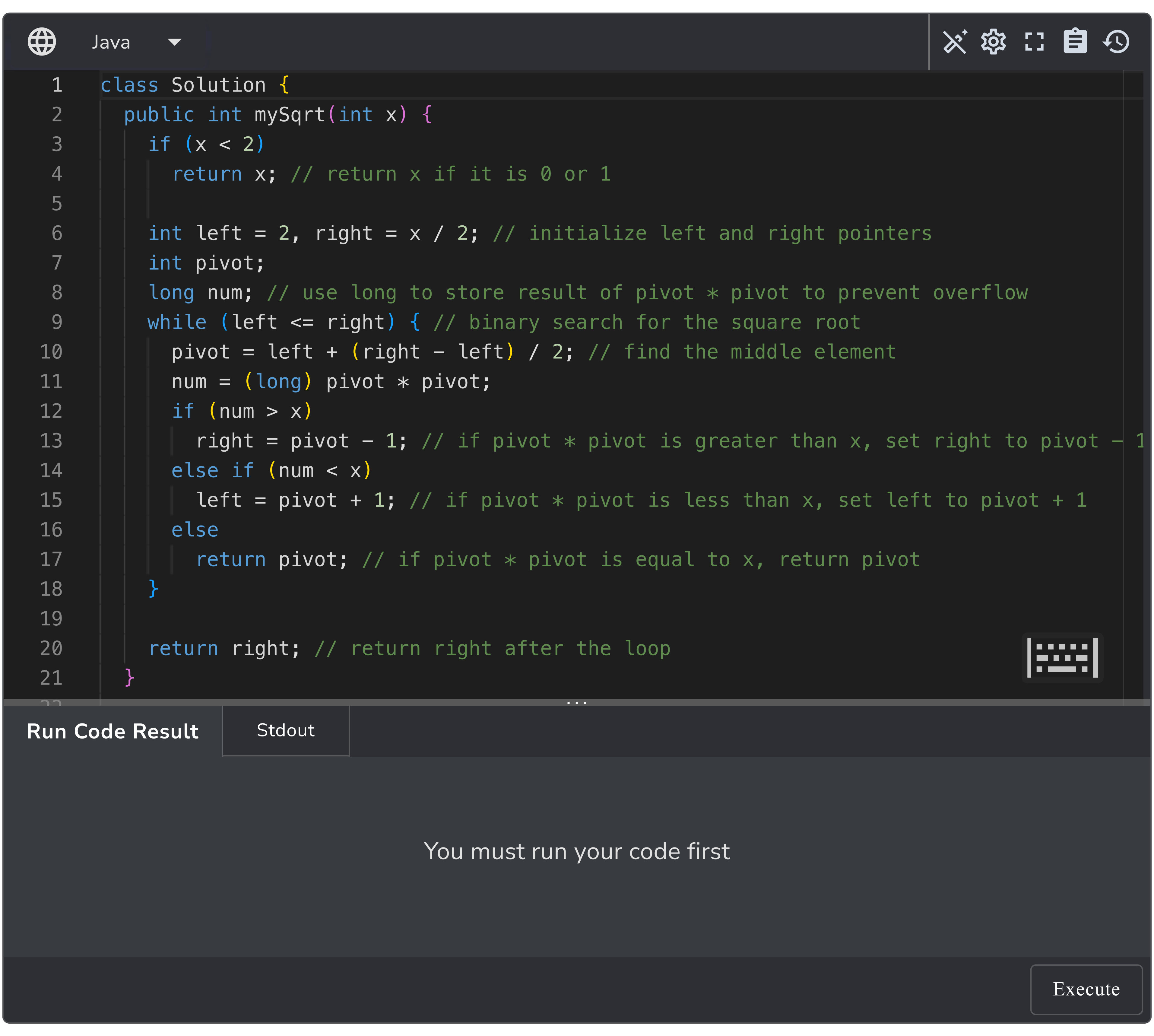
Walk-through of the algorithm

Now let's walk through the code:

1. We first handle the base case. If the input number 'x' is less than 2, we return 'x' itself because the square root of 0 is 0 and the square root of 1 is 1.
2. Then, we initialize two pointers, 'left' and 'right'. The 'left' pointer is set to 2 and the 'right' pointer is set to x/2. These pointers define the range within which we will search for the square root.
3. We then enter a `while` loop, which continues while 'left' remains less than or equal to 'right'.
4. In each iteration, we calculate the 'pivot' which is the middle element between 'left' and 'right'. The 'pivot' essentially represents our current guess for the square root.
5. We calculate 'num', which is the square of the 'pivot'. Since squaring a number can lead to overflow for large numbers, we use a 'long' data type for 'num'.
6. Next, we compare 'num' with 'x':
    - If 'num' is greater than 'x', it means our 'pivot' is too large. So, we reduce our 'right' pointer to 'pivot - 1' to search in the lower half.
    - If 'num' is less than 'x', it means our 'pivot' is too small. So, we increase our 'left' pointer to 'pivot + 1' to search in the upper half.
    - If 'num' equals 'x', it means we've found the exact square root, so we return 'pivot'.
7. If we exit the while loop without returning (which means we didn't find an exact square root), we return 'right' as our final result, which will be the largest integer less than or equal to the square root of 'x'.

Code

Here is the code for this algorithm:



```java
class Solution {
  public int mySqrt(int x) {
    if (x < 2)
      return x; // return x if it is 0 or 1

    int left = 2, right = x / 2; // initialize left and right pointers
    int pivot;
    long num; // use long to store result of pivot * pivot to prevent overflow
    while (left <= right) { // binary search for the square root
      pivot = left + (right - left) / 2; // find the middle element
      num = (long) pivot * pivot;
      if (num > x)
        right = pivot - 1; // if pivot * pivot is greater than x, set right to pivot - 1
      else if (num < x)
        left = pivot + 1; // if pivot * pivot is less than x, set left to pivot + 1
      else
        return pivot; // if pivot * pivot is equal to x, return pivot
    }

    return right; // return right after the loop
  }
```

You must run your code first

Time Complexity

1. **Binary Search Algorithm:** The key part of this algorithm is the binary search, which repeatedly divides the search interval in half. The time complexity of binary search is $O(logn)$, where n is the size of the search space. In this case, the search space is initially from `2` to `x/2`.
2. **Search Space:** The maximum size of the search space is `x/2` (when $x \geq 4$). For smaller values of `x`, the function immediately returns `x`, as it's either `0` or `1`.
3. **Overall Time Complexity:** Considering the binary search on a range up to `x/2`, the time complexity is $O(log(x/2))$, which simplifies to $O(\log x)$.

Space Complexity

1. **Constant Extra Space:** The algorithm uses a fixed number of integer variables (`left`, `right`, `pivot`, `num`), regardless of the input size.
2. **No Recursive Calls or Dynamic Allocation:** The implementation does not use recursion or allocate additional data structures that grow with the input size.
3. **Overall Space Complexity:** Given the constant amount of extra space, the space complexity is $O(1)$, meaning it's constant.

Conclusion

- **Time Complexity:** $O(\log x)$
- **Space Complexity:** $O(1)$ (Constant space)

← Previous
Sqrt (Medium)

Next →
Introduction To Two Pointers Pattern

☐ Mark as Completed